ENGINEERING DECISION · ASYNCHRONOUS WORK · ONE-PAGE BRIEF

A queue is not extra capacity.

A fictional report service moves generation behind a queue. Requests return quickly, but the workers do the same work at the same rate: the waiting moved, the work did not.

1,200
extra unfinished jobs after a 30-second burst[1]

40/s
recovery headroom once arrivals fall[1]

30 s
to drain the backlog after the burst ends[1]

Only spare capacity drains the backlog

While arrivals exceed completions, unfinished work grows. Afterward, only the completion rate left over after new arrivals drains it. **At full utilization the backlog stays.**

Watch the oldest unfinished promise

Queue depth counts work; **job age exposes delay**. Track age from acceptance through completion, including in-flight jobs, and alert while there is still time to intervene.

Retries are more arrivals

Give each logical job one stable identity and a **finite retry budget**. Backoff spreads duplicate demand over time; it does not make the work free.

Bound the promise, not just the queue

Combine a hard storage cap with an admission estimate of waiting plus execution. Reject clearly when either budget is exhausted. **Every accepted job ends** completed, failed, or explicitly expired.

Decision

- Use a bounded queue with explicit admission and job status.
- Add workers only when the downstream bottleneck has room; buffer more only when the waiting budget allows it.
- Rehearse a burst, a slow dependency, and a worker restart with normal traffic running; account for every accepted job.

Buffer the burst. Budget the recovery.

[1] Fluid approximation with invented inputs: 80 jobs/s completion capacity; 120 jobs/s arrivals for 30 s, then 40 jobs/s; no retries, expiry, or failures. Not production measurements. The reasoning, assumptions, and rehearsal criteria are in the interactive document.